

Project 01: The Searchin' Pac-Man

Search implementation and performance report

Student name	Stony Brook ID
Param Shah	117369485
Madhura Chitale	117420795

Scope and implementation

We implemented Questions 1-7 in `search.py` and `searchAgents.py`. We retained the supplied function and class names, framework code, expansion counters, and Berkeley attribution. The implementation covered DFS, BFS, UCS, A*, the corners problem, and the two heuristics. The following sections documented the measurements and answered the assignment discussion questions.

Implementation summary

DFS used a stack and tracked expanded states. BFS used a FIFO queue and tracked states when they were discovered. UCS reused A* with a zero heuristic. A* prioritized entries by $g+h$, recorded the best known path costs, skipped outdated queue entries, and allowed a state to reopen when a cheaper route was found. The search functions returned action lists. Corners states contained position and immutable corner flags, while food states used the supplied position-and-food-grid representation.

Timing methodology

The 28 benchmark cases were measured on September 10, 2026 using CPython 3.10.10 on Windows 10.0.26200. Each reported runtime was the median of three fresh searches measured with `time.perf_counter`. Each run began with a new problem and empty heuristic caches, and garbage collection ran before timing. Imports, layout loading, problem construction, graphics, and plan execution were excluded. The assignment recommended Python 3.11, but that version was not available for these measurements.

Memory and expansion methodology

Peak memory was measured in a separate fresh search using `tracemalloc`. The measurement included search allocations, bookkeeping, and heuristic caches, but excluded pre-existing interpreter and maze allocations. It represented traced Python memory rather than total process memory; one KiB equaled 1024 bytes. Node counts were read from `problem._expanded` before plan replay. Auxiliary BFS work inside the heuristics contributed to runtime and memory, but was not included in this expansion counter.

Reading the results

The tables included all 28 benchmark cases and reported memory to two decimal places. Path length and path cost were equal for unit-cost problems; weighted costs were reported separately. Runtime comparisons were interpreted within the measured environment. The optional `mediumSearch` trial was performed on September 16 and was reported separately from the 28 benchmark cases.

Q1: DFS and fixed-position measurements

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyMaze	DFS	10	15	0.074	3.93
tinyMaze	BFS	8	15	0.071	3.85
tinyMaze	UCS	8	15	0.077	4.20
tinyMaze	A* Manhattan	8	14	0.083	4.22
mediumMaze	DFS	130	146	0.581	30.89
mediumMaze	BFS	68	269	0.974	38.95
mediumMaze	UCS	68	269	1.106	41.08
mediumMaze	A* Manhattan	68	221	0.997	38.55
bigMaze	DFS	210	390	1.581	116.82
bigMaze	BFS	210	620	2.452	103.26
bigMaze	UCS	210	620	2.657	90.11
bigMaze	A* Manhattan	210	549	2.714	88.81
openMaze	DFS	298	576	3.229	562.52
openMaze	BFS	54	682	2.642	99.49
openMaze	UCS	54	682	3.106	123.12
openMaze	A* Manhattan	54	535	2.466	85.63

Q1 - Is the exploration order what we expect?

The implementation followed the expected LIFO exploration order. It pushed legal successors in North, South, East, West order and processed the most recently pushed entry first. When all four successors were eligible, West was processed next. Search continued down a branch before considering alternatives and skipped states that had already been expanded. This assessment was based on the code; the graphical expansion overlay was not separately inspected.

Q1 - Does Pac-Man visit every explored square?

Pac-Man did not need to visit every explored square. Expansion represented planning work, whereas movement followed only the returned action list. States on abandoned search branches therefore did not have to appear in the executed route. On mediumMaze, DFS returned a 130-move route after expanding 146 states; those values measured different aspects of the search.

Q1 - Is DFS least-cost, and why does order matter?

DFS did not find a least-cost route on mediumMaze: it returned 130 moves, compared with 68 for BFS. It prioritized depth rather than accumulated cost and stopped at the first goal selected from the stack. Its traversal depended on successor insertion order. The 130-step result differed from the assignment's approximate examples even though the supplied order was preserved. The cause of that discrepancy was not established.

Q2-Q4: BFS, weighted UCS, and A*

Q2 - Does BFS find a least-cost solution?

BFS found least-cost solutions on the tested unit-cost position mazes. It returned 8, 68, 210, and 54 moves on tinyMaze, mediumMaze, bigMaze, and openMaze, respectively. FIFO ordering explored states by increasing depth, and the discovered set prevented repeated enqueueing and expansion. This result depended on equal move costs; minimizing the number of moves alone would not have guaranteed minimum cost on the weighted layouts.

Q2 - Does the generic code work for the eight-puzzle?

BFS, UCS, and A* solved the supplied one-move eight-puzzle instance without changes to search.py. The supplied EightPuzzleSearchProblem.getStartState method read a module-level puzzle variable rather than self.puzzle. The validation setup initialized that variable in the same way as the supplied main program. No puzzle-specific logic was added to the search algorithms.

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
mediumDottedMaze	UCS east	74	186	0.987	41.12
mediumScaryMaze	UCS west	92	108	0.609	30.14

Q3 - Why are the costs very low and very high?

StayEast charged $0.5 \cdot x$ for entering column x , which made eastern positions cheaper. Its 74-move route cost 1.000976583804004. StayWest charged $2 \cdot x$, which made eastern positions more expensive; its 92-move route cost 68,719,479,864. Both headless games ended in wins. UCS prioritized accumulated cost rather than move count. Calling A* with a zero heuristic produced this UCS ordering.

Q4 - Does A* match the bigMaze comparison?

Manhattan A* found the same 210-step optimum as UCS and expanded 549 states rather than 620, matching the assignment's reference counts. Its measured runtime was 2.714 ms, compared with 2.657 ms for UCS. The reduction in expansions did not produce a runtime improvement in this small measurement. Manhattan distance replaced the null default, and entries were prioritized by $f=g+h$.

Q4 - What happens on openMaze?

DFS returned 298 moves, expanded 576 states, and used 562.52 KiB. BFS and UCS each returned the 54-step optimum and expanded 682 states. Manhattan A* also returned 54 moves, but expanded 535 states and took 2.466 ms, compared with 3.106 ms for UCS. Its peak memory was 85.63 KiB. DFS used substantially more memory because its frontier stored complete action prefixes and retained pending alternatives.

Q5-Q6: visiting all corners

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyCorners	BFS	28	252	1.419	85.07
tinyCorners	A* corners	28	159	3.009	84.34
mediumCorners	BFS	106	1966	11.189	678.29
mediumCorners	A* corners	106	741	19.303	317.91

Q5 - What is the state, and why is it sufficient?

Each state contained Pac-Man's position and immutable visited-corner flags. This representation recorded the information needed to complete the task without storing ghosts, score, or unrelated food. The problem object stored the walls and fixed corner locations. A corner occupied initially was marked visited. Immutable tuples kept alternative search branches independent and supported set and dictionary membership checks.

Q5 - How are successors, goals, and costs defined?

Each legal north, south, east, or west move produced a new position and flag tuple at cost 1. Entering a corner set its flag, and previously set flags remained true. The goal test required all four flags to be true, regardless of whether food existed at the corners. The supplied action-cost method checked legality and counted moves. BFS found 28 steps on tinyCorners and 106 on mediumCorners, expanding 1,966 states on mediumCorners.

Q6 - What heuristic is used?

The heuristic computed the shortest Manhattan route through all unvisited corners without returning to the start. It considered all remaining visiting orders recursively, with at most $4! = 24$ complete orders. It selected the best total route rather than greedily choosing the nearest corner. This calculation solved the wall-free relaxation exactly, while avoiding an exact solution of the original maze problem.

Q6 - Is the heuristic admissible and consistent?

The admissibility argument followed from removing walls: the relaxed optimum could not exceed the real completion cost. A legal move followed by an optimal relaxed route from the successor also formed a candidate relaxed route from the original state. This established $h(s) \leq 1 + h(s')$, the consistency condition. The implementation returned nonnegative values and zero when no corners remained. It provided a nontrivial bound without solving the original completion problem exactly.

Q6 - What is the performance and scoring implication?

A* retained the 106-step optimum and reduced mediumCorners expansions from 1,966 to 741. This was below the 1,200-node full-credit threshold and the 800-node stretch marker, which carried no additional stated points. Peak memory decreased from 678.29 to 317.91 KiB. Runtime increased from 11.189 to 19.303 ms, so evaluating the heuristic outweighed the benefit of fewer expansions in this measurement.

Q7: collecting all food efficiently

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
testSearch	UCS	7	14	0.419	16.75
testSearch	A* food MST	7	7	0.285	13.08
tinySearch	UCS	27	5057	219.103	8400.16
tinySearch	A* food MST	27	89	5.945	264.44
trickySearch	UCS	60	16688	1485.511	48296.31
trickySearch	A* food MST	60	255	30.424	1065.58

Q7 - What is implemented?

The supplied FoodSearchProblem represented each state by position and a remaining-food grid. It copied the grid for each successor, removed food at the destination, and assigned move cost 1. The implemented heuristic added the nearest-food maze distance to the minimum spanning tree (MST) weight over the remaining food. Edge weights were exact maze distances, and Prim's algorithm computed the MST. The tree supplied a lower bound rather than an executable route.

Q7 - Why BFS and caching inside the heuristic?

BFS from each food coordinate computed shortest distances to reachable squares because every move had the same cost. The implementation reused these maps because the walls remained unchanged. It cached MST costs by frozenset(food), allowing states with the same remaining food to reuse a tree cost despite different Pac-Man positions. These auxiliary searches operated over positions, rather than solving the full food-collection problem.

Q7 - Do A* and UCS agree, and what are the node thresholds?

Both methods returned costs of 7 on testSearch, 27 on tinySearch, and 60 on trickySearch. On trickySearch, A* expanded 255 states, compared with 16,688 for UCS. The assignment awarded 5 points below 15,000 nodes, 15 below 12,000, 20 below 9,000, and 5 extra points below 7,000, subject to heuristic correctness. The measured result met the highest threshold, but no official grade was obtained.

Q7 - Can mediumSearch be solved in a short time?

A separate headless AStarFoodSearchAgent trial on September 16 was limited to 30 seconds. It did not finish within that limit and was terminated. The trial therefore did not demonstrate a short-time solution, and no solution cost or final expansion count was obtained. This optional challenge had no stated scoring threshold and was kept separate from the median benchmark timings.

Q7: admissibility, consistency, and validation

Why nearest food plus the MST is admissible

The proof used p for Pac-Man's position, F for remaining food, $d(p,F)$ for nearest-food distance, and $T(F)$ for MST weight. The heuristic was $h(p,F)=d(p,F)+T(F)$. Every completion required an initial journey to food, with cost at least $d(p,F)$. The subsequent first-visit order defined a spanning path whose maze-distance weight did not exceed the remaining travel. The MST weighed no more than that path. These bounds applied to successive portions of the route, so their sum did not overestimate completion cost.

Consistency when a move does not eat food

For a unit move from p to p' , the food set F remained unchanged, so the MST term stayed the same. The triangle inequality gave $d(p,F) \leq 1 + d(p',F)$. Adding the unchanged tree weight established $h(p,F) \leq 1 + h(p',F)$. Thus a move without eating food could not reduce the heuristic by more than its cost.

Consistency when a move eats food

Suppose a move reached and ate dot a , leaving food set R . Before the move, nearest-food distance was at most 1. If R was nonempty, attaching a to its nearest dot in an MST of R gave a candidate tree for the old food set. Therefore $T(F) \leq d(a,R)+T(R)$, which established $h(p,F) \leq 1+h(a,R)$. If a was the last dot, the heuristic changed from 1 to 0. The goal case and both transition cases satisfied consistency.

What should happen to f-values along a path?

For a successor on the same path, $g(s')=g(s)+1$. Combining this equation with consistency gave $g(s)+h(s) \leq g(s')+h(s')$. Consequently, f-values were nondecreasing along a path. The implementation also allowed reopening after discovering a cheaper route. Although reopening supported admissible but inconsistent heuristics, the food heuristic was designed to satisfy the assignment's stronger consistency requirement.

What if UCS and A* have different solution costs?

Different costs would have required investigation of the heuristic, search implementation, and comparison setup. Agreement on examples alone would not have proved admissibility. The mathematical argument established the bound, while exhaustive checks verified consistency across 912 transitions over 384 tinyCorners states and 17,732 transitions over 8,100 tinySearch states. Checked values were nonnegative and zero at goals, and tested A* and UCS solution costs agreed.

Other checks and limits

All 28 benchmark plans passed legal-action replay and reached their goals. Additional checks covered start-at-goal, unreachable goals, cycles, cheaper duplicate routes, A* reopening, eight-puzzle search, starting at a corner, and parent-state independence. Source compilation and representative headless games passed. The package contained grading support code but lacked autograder.py and test_cases, so no official autograder result was obtained.

Critical analysis and conclusion

Critical assessment - solution quality and memory

DFS reached some goals after relatively few expansions, but its preference for depth produced longer routes on mediumMaze and openMaze. BFS provided a reliable baseline for unit-cost shortest paths, while UCS handled the weighted layouts. Storing full action lists increased memory use: DFS required 562.52 KiB on openMaze, compared with 99.49 KiB for BFS. The measurements reflected this implementation's copied paths and global state tracking.

Critical assessment - heuristic cost versus benefit

Manhattan A* reduced expansions but did not improve runtime on every tested maze. The corner heuristic reduced mediumCorners expansions and memory use, but ran more slowly than BFS. The food MST produced the largest measured improvement. On trickySearch, expansions fell from 16,688 to 255, runtime fell from 1,485.511 to 30.424 ms, and peak memory fell from 48,296.31 to 1,065.58 KiB. UCS and food A* both retained the 60-step optimum.

Limitations of the evaluation

The mediumSearch trial did not finish within 30 seconds, so the results did not establish efficient performance on larger food problems. Python 3.11 and the official autograder were unavailable during validation. Runtime and memory measurements described the tested environment, and they were not treated as universal performance guarantees. These limitations did not invalidate the measured results, but they restricted the conclusions drawn from them.

Overall assessment

The experiments showed that no single search strategy performed best across all measured criteria. BFS found shortest paths efficiently on unit-cost mazes, and UCS accommodated varying step costs. A* reduced exploration when its heuristic provided useful guidance, although heuristic computation sometimes increased runtime. The food MST achieved the strongest improvement on the tested food mazes. The results supported comparing solution cost, node expansions, runtime, and memory together rather than relying on one statistic alone.